

Atividade Lab - Computação Distribuída com MPI: Multiplicação de Matrizes e Método de Monte Carlo

MPI + Python + Docker

Prof Alcides/Prof Mário

1. Arquitetura, Funções e Tipos de Dados em MPI (Python)

1.1 Estrutura Básica de um Programa MPI

Uma aplicação MPI (*Message Passing Interface*) é composta por múltiplos processos independentes que executam o mesmo programa simultaneamente. Cada processo recebe um identificador numérico único denominado **rank** ($0, 1, \dots, size - 1$). Esse identificador permite que cada processo execute comportamentos ou subcálculos distintos dentro da mesma base de código.

Em Python, a integração com o padrão MPI é fornecida de forma eficiente pela biblioteca `mpi4py`.

```
from mpi4py import MPI

# 1. Inicializa o ambiente de execução MPI (gerenciado ao importar mpi4py)
comm = MPI.COMM_WORLD

# Obtenção do tamanho do comunicador e do rank do processo atual
size = comm.Get_size()
rank = comm.Get_rank()

print(f"Processo {rank} de {size}")

# 2. Finaliza o ambiente MPI (opcional; mpi4py finaliza automaticamente ao encerrar)
# MPI.Finalize()
```

Onde:

- **size**: número total de processos instanciados no comunicador.
- **rank**: identificador exclusivo do processo atual.

Observação: Em Python, a inicialização do ambiente ocorre automaticamente ao importar `mpi4py`. O uso explícito de `MPI.Init()` e `MPI.Finalize()` é recomendado apenas quando se deseja um controle estrito sobre o ciclo de vida do ambiente MPI.

Preparação da Infraestrutura Docker e Execução

Para executar programas MPI em um cluster distribuído simulado por containers Docker, utilize o procedimento descrito a seguir:

```
# 1. Inicialização dos containers e serviço SSH
docker compose up -d
docker compose exec master service ssh start
docker compose exec worker1 service ssh start
docker compose exec worker2 service ssh start
docker compose exec worker3 service ssh start

# 2. Instalação das dependências (pip e mpi4py) nos nós do cluster
docker compose exec master apt update && docker compose exec master apt install -y python3-pip
docker compose exec worker1 apt update && docker compose exec worker1 apt install -y python3-pip
docker compose exec worker2 apt update && docker compose exec worker2 apt install -y python3-pip
docker compose exec worker3 apt update && docker compose exec worker3 apt install -y python3-pip

docker compose exec master python3 -m pip install mpi4py
docker compose exec worker1 python3 -m pip install mpi4py
docker compose exec worker2 python3 -m pip install mpi4py
docker compose exec worker3 python3 -m pip install mpi4py

# 3. Transferência do código-fonte para todos os nós
docker cp programa.py master:/home/mpiuser/
docker cp programa.py worker1:/home/mpiuser/
docker cp programa.py worker2:/home/mpiuser/
docker cp programa.py worker3:/home/mpiuser/

# 4. Execução do programa MPI a partir do host (Codespaces)
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 programa.py"
```

1.2 Funções Fundamentais e Seus Parâmetros

- **MPI.COMM_WORLD**: Comunicador global que agrupa todos os processos participantes da execução.
 - **comm.Get_size()**: Retorna o número total de processos presentes no comunicador.
 - **comm.Get_rank()**: Retorna o identificador exclusivo (*rank*) do processo chamador.
 - **MPI.Init()** / **MPI.Finalize()**: Funções opcionais para controle manual do ciclo de vida da execução MPI.
-

1.3 Funções de Comunicação Ponto a Ponto

A comunicação ponto a ponto permite a transferência direta de dados entre dois processos distintos (emissor e receptor).

```
# Envio bloqueante de objeto Python
comm.send(obj, dest, tag=0)

# Recepção bloqueante de objeto Python
data = comm.recv(source=MPI.ANY_SOURCE, tag=MPI.ANY_TAG, status=None)
```

Parâmetros e variantes principais:

- **comm.send(obj, dest, tag=0)**: Envia qualquer objeto Python serializável para o processo **dest**.
 - **comm.recv(source, tag, status)**: Recebe uma mensagem proveniente de **source**. O parâmetro opcional **status = MPI.Status()** permite inspecionar metadados da mensagem recebida (rank de origem real, tag e tamanho do conteúdo).
 - **comm.ssend(obj, dest, tag=0)**: Envio síncrono. Garante que a função só retorne quando o receptor tiver iniciado a chamada **recv** correspondente.
 - **comm.sendrecv(...)**: Executa o envio e a recepção de dados em uma única operação atômica, prevenindo situações de *deadlock* em trocas bidirecionais.
-

1.4 Tipos de Dados em MPI (Python)

A biblioteca `mpi4py` disponibiliza dois paradigmas para movimentação de dados:

1. **Alto Nível (Objetos Python Genéricos)**: Utiliza inicial minúscula nas funções (**send**, **recv**, **bcast**, **scatter**, **gather**, **reduce**). Aceita qualquer objeto Python serializável (via `pickle`). Apresenta excelente facilidade de desenvolvimento, mas introduz *overhead* de serialização.
 2. **Baixo Nível (Buffers Nativos de Desempenho)**: Utiliza inicial maiúscula nas funções (**Send**, **Recv**, **Bcast**, **Scatter**, **Gather**, **Reduce**). Atua diretamente sobre buffers de memória contínua (ex.: `arrays numpy.ndarray`). Elimina a serialização e maximiza a vazão de rede.
-

1.5 Tabela de Referência de Mapeamento de Tipos

MPI Datatype (C)	Python Alto Nível (mpi4py)	Python Bufferizado (NumPy + mpi4py)
MPI_CHAR	str / bytes	np.ndarray(dtype='int8') + MPI.CHAR
MPI_INT	int	np.ndarray(dtype='int32') + MPI.INT
MPI_LONG	int	np.ndarray(dtype='int64') + MPI.LONG
MPI_FLOAT	float	np.ndarray(dtype='float32') + MPI.FLOAT
MPI_DOUBLE	float	np.ndarray(dtype='float64') + MPI.DOUBLE
MPI_BYTE	bytes	np.ndarray(dtype='uint8') + MPI.BYTE

1.6 Tipos de Dados Derivados em Python

Enquanto em C é necessário construir tipos de dados derivados com `MPI_Type_create_struct`, em Python a maneira idiomática e de alto desempenho consiste em utilizar *views* e *slices* do NumPy sobre matrizes e estruturas contíguas de memória.

2. Padrões de Comunicação Coletiva em MPI

As operações coletivas envolvem simultaneamente todos os processos pertencentes a um comunicador.

2.1 Broadcast (`comm.bcast`)

Transmite um dado localizado no processo *root* para todos os demais processos do comunicador.

```
dados = comm.bcast(dados_originais, root=0)
```

Após a execução, todos os nós possuem uma cópia idêntica de `dados`. Padrão comum para distribuição de parâmetros globais e matrizes de entrada.

2.2 Gather (`comm.gather`)

Reúne valores produzidos por todos os processos em uma lista centralizada no processo *root*.

```
resultados = comm.gather(valor_local, root=0)
```

O processo *root* recebe uma lista contendo os dados de todos os ranks em ordem; os demais processos recebem `None`.

2.3 Reduce (`comm.reduce`)

Combina os valores locais de cada processo através de uma operação matemática acumuladora entregando o resultado final ao processo *root*.

```
total = comm.reduce(valor_local, op=MPI.SUM, root=0)
```

Operações de redução predefinidas:

- `MPI.SUM`: Soma de todos os valores.
 - `MPI.MAX`: Valor máximo.
 - `MPI.MIN`: Valor mínimo.
 - `MPI.PROD`: Produto acumulado.
-

3. Exemplo Prático: “Olá Mundo” Distribuído

Abaixo é apresentado um exemplo de comunicação ponto a ponto onde o processo *master* (rank 0) coleta mensagens enviadas por todos os trabalhadores (*workers*).

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()

if rank != 0:
    mensagem = f"Saudações do processo {rank} de {size}"
    comm.send(mensagem, dest=0)
else:
    print(f"Saudações do processo {rank} de {size}")
    for i in range(1, size):
        mensagem = comm.recv(source=i)
        print(mensagem)
```

4. Multiplicação de Matrizes (Sequencial e Multithreaded)

A multiplicação de matrizes é um problema clássico para estudo de paralelismo, pois o cálculo de cada elemento ou linha da matriz resultante C é independente dos demais.

Dadas duas matrizes quadradas A e B de dimensão $N \times N$, a matriz resultante $C = A \times B$ é definida por:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

4.1 Implementação Sequencial (Python Local)

```
import random
import time

N = 300
A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0]*N for _ in range(N)]

inicio = time.time()
for i in range(N):
    for j in range(N):
        for k in range(N):
            C[i][j] += A[i][k] * B[k][j]
fim = time.time()

print(f"Tempo sequencial: {(fim - inicio) * 1000:.2f} ms")
```

A complexidade computacional deste algoritmo sequencial é $\mathcal{O}(N^3)$.

4.2 Implementação com Threads (Python Local)

Nesta versão, o trabalho de calcular as N linhas da matriz C é particionado entre múltiplas *threads* em uma única máquina.

```
import threading
import random
import time

N = 300
THREADS = 4

A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0]*N for _ in range(N)]
```

```

def calcular(inicio, fim):
    for i in range(inicio, fim):
        for j in range(N):
            for k in range(N):
                C[i][j] += A[i][k] * B[k][j]

inicio = time.time()
threads = []
linhas_por_thread = N // THREADS

for t in range(THREADS):
    ini = t * linhas_por_thread
    fim_t = N if t == THREADS - 1 else (t + 1) * linhas_por_thread
    th = threading.Thread(target=calcular, args=(ini, fim_t))
    threads.append(th)
    th.start()

for th in threads:
    th.join()

fim = time.time()
print(f"Tempo com threads: {(fim - inicio) * 1000:.2f} ms")

```

5. Exercícios Práticos Avaliativos

Exercício 1 – Multiplicação de Matrizes Distribuída com MPI

Desenvolva uma versão distribuída do algoritmo de multiplicação de matrizes utilizando a biblioteca `mpi4py` para rodar no cluster de 4 containers Docker.

Especificação e Regras

1. **Geração das Matrizes:** O processo `rank 0` deve gerar aleatoriamente as matrizes A e B de dimensão $N \times N$.
2. **Distribuição dos Dados:** As matrizes A e B devem ser transmitidas a todos os processos do comunicador através da operação coletiva `comm.bcast`.
3. **Cálculo Distribuído:** Cada processo r calculará apenas a sua fatia de linhas correspondente da matriz resultado C .
4. **Coleta dos Resultados:** As partes calculadas por cada nó devem ser enviadas de volta ao processo 0 utilizando a rotina `comm.gather`.
5. **Consolidação e Tempo:** O processo 0 montará a matriz C final e imprimirá o tempo total de execução em milissegundos.

Funções MPI Recomendadas

- `comm.Get_rank()`
- `comm.Get_size()`
- `comm.bcast(objeto, root=0)`
- `comm.gather(objeto_local, root=0)`

Experimentos e Benchmarking

Execute o experimento para as seguintes dimensões de matriz N :

- **Teste 1:** $N = 300$
- **Teste 2:** $N = 600$
- **Teste 3:** $N = 1000$

Preencha a tabela de desempenho abaixo com os tempos medidos em milissegundos e anexe-a ao relatório final da atividade:

Dimensão (N)	Tempo Sequencial (ms)	Tempo Multithreaded (ms)	Tempo Distribuído MPI (ms)
$N = 300$			
$N = 600$			
$N = 1000$			

Exercício 2 – Estimativa do Valor de π utilizando o Método de Monte Carlo

O Método de Monte Carlo utiliza amostragem aleatória estatística para estimar grandezas numéricas. Neste exercício, estimaremos o valor da constante π .

Fundamentação Matemática

Considere o primeiro quadrante de um plano cartesiano com um quadrado de lado 1 contendo um quarto de círculo de raio $r = 1$ inscrito.

Se gerarmos N_{total} pontos aleatórios (x, y) uniformemente distribuídos em $[0, 1] \times [0, 1]$:

- A probabilidade de um ponto cair no interior do quarto de círculo é igual à razão entre a área do quarto de círculo ($\frac{\pi r^2}{4} = \frac{\pi}{4}$) e a área do quadrado ($1^2 = 1$).
- Assim:

$$\frac{\text{Pontos dentro do Círculo}}{\text{Total de Pontos}} \approx \frac{\pi}{4} \Rightarrow \pi \approx 4 \cdot \frac{\text{Pontos dentro do Círculo}}{\text{Total de Pontos}}$$

Passos para a Solução Distribuída

1. **Particionamento:** Divida o total de pontos N igualmente entre os $size$ processos instanciados. Cada processo gerará $N_{local} = N/size$ pontos.
2. **Geração e Contagem Local:** Cada processo gera N_{local} pares aleatórios (x, y) com $0 \leq x, y \leq 1$ e incrementa seu contador local sempre que $x^2 + y^2 \leq 1$.
3. **Agregação de Resultados:** Utilize a operação coletiva `comm.reduce` com o operador `MPI.SUM` para somar a quantidade total de pontos internos no processo `rank 0`.
4. **Cálculo Final:** O processo 0 calcula a estimativa de π utilizando a fórmula e exibe o resultado final juntamente com o tempo total de execução.

Código Sequencial de Referência (Local)

```
import random
import time

N = 10_000_000
dentro = 0

inicio = time.time()
for _ in range(N):
    x = random.random()
    y = random.random()
    if x * x + y * y <= 1.0:
        dentro += 1

pi_estimado = 4.0 * dentro / N
fim = time.time()

print(f"PI aproximado: {pi_estimado:.6f}")
print(f"Tempo sequencial: {(fim - inicio) * 1000:.2f} ms")
```

6. Diretrizes para Elaboração do Relatório de Laboratório

O relatório final é um elemento obrigatório de avaliação da atividade prática. Ele deve ser elaborado com rigor acadêmico e clareza técnica, seguindo a estrutura padrão adotada em laboratórios de programação avançada e computação distribuída.

6.1 Formato de Entrega

- **Arquivo:** Exclusivamente em formato **PDF**.
- **Identificação:** Nome completo, RA, turma, disciplina e data de entrega na primeira página.

6.2 Estrutura Obrigatória do Relatório

1. Capa / Identificação e Título:

- Identificação dos integrantes, instituição (FCI - Mackenzie) e título do laboratório.

2. Introdução e Fundamentação Teórica:

- Apresentação sucinta dos conceitos de comunicação coletiva em MPI (**bcast**, **gather**, **reduce**), divisão de tarefas (*data partitioning*) e estimativa por Monte Carlo.

3. Metodologia e Ambiente de Execução:

- Descrição do ambiente computacional utilizado (ex.: Codespaces, Docker Desktop, Linux/macOS/Windows, versão do Python e **mpi4py**).
- Descrição da topologia do cluster simulado (Master + 3 Workers).

4. Resultados e Evidências de Execução (Prints Obrigatórios):

- **Tabelas de Desempenho:** Preenchimento completo das tabelas comparativas para os Exercícios 1 ($N = 300, 600, 1000$) e 2 ($N = 10.000.000$).
- **Capturas de Tela (*Screenshots/Prints*):** É **obrigatório** incluir capturas de tela claras e legíveis do terminal demonstrando:
 - Subida dos containers Docker e inicialização do cluster.
 - Execução do comando **mpirun** no processo *master*.
 - Saídas de terminal impressas contendo os tempos calculados e os resultados (C acumulado e estimativa de π).

5. Análise de Desempenho e Discussão de Resultados:

- Comparação crítica do tempo de execução entre as versões Sequencial, Multithreaded e Distribuída com MPI.
- Discussão sobre o ganho de desempenho (*speedup*), gargalos de comunicação (*overhead* de rede/serialização de objetos em **mpi4py**) e quando o uso de cluster distribuído se justifica em relação ao uso de *threads* locais.

6. Seção de Dificuldades Encontradas e Soluções:

- Descrição detalhada dos eventuais problemas técnicos enfrentados durante o desenvolvimento (ex.: erros de conectividade SSH nos containers, divergência em dimensões de matrizes, inicialização do comunicador, incompatibilidade de bibliotecas ou problemas na instalação do **mpi4py**).

- Apresentação clara de como essas dificuldades foram superadas e as soluções adotadas.

7. Seção Obrigatória sobre o Uso de Inteligência Artificial:

- Declaração transparente quanto ao uso de ferramentas de Inteligência Artificial generativa (ex.: ChatGPT, Claude, Gemini, GitHub Copilot, etc.) durante a realização do trabalho.
- Especificação detalhada de:
 - Quais ferramentas foram consultadas e os *prompts* principais utilizados.
 - Em quais etapas da atividade a IA auxiliou (ex.: depuração de sintaxe, entendimento de erro do Docker/MPI, formatação do relatório, explicação de conceito).
 - **Análise Crítica:** Avaliação reflexiva do estudante sobre a precisão das respostas fornecidas pela IA, indicando eventuais alucinações, respostas incorretas que precisaram de correção humana e o aprendizado consolidado pelo próprio aluno.

8. Conclusão e Referências:

- Síntese dos aprendizados obtidos com a atividade prática.
- Referências bibliográficas (documentação oficial da biblioteca `mpi4py`, documentação do MPI standard e materiais didáticos da disciplina).

Instruções de Submissão e Execução no Cluster Docker

Para compilar, implantar e executar as soluções dos Exercícios 1 e 2 no cluster, siga os comandos:

```
# 1. Copiar o script do exercício para os nós do cluster
docker cp exercicio1.py master:/home/mpiuser/
docker cp exercicio1.py worker1:/home/mpiuser/
docker cp exercicio1.py worker2:/home/mpiuser/
docker cp exercicio1.py worker3:/home/mpiuser/

# 2. Executar via SSH no master com 4 processos distribuídos
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 exercicio1.py"
```

Repita o procedimento para `exercicio2.py`. Entregue o relatório em PDF com o código fonte e as evidências de execução.